**SIMATS ENGINEERING**
**ASSESSMENT TOOL 2**

**COURSE NAME: INTERNET PROGRAMMING**
**COURSE CODE: CSA43**

**A. MONESH (192311430)**

**COURSE OUTCOME COVERED**

**CO-1:** Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool : Data Interpretation

Weightage: 20%

**QUESTIONS:** **Total Marks: 20**

1: HTTP Response Analysis

A web server returns the following HTTP response header:
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive **Questions:**

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with Content-Length and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
  <head>
    <title>Test Page</title>
  </head>
  <body>
    <div>
      <p>Hello World
    </div>
  </body>
</html>
```

**Questions:**

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

| Method | URL Example | Data Visibility | Security Level |
|---|---|---|---|
| GET | /login?user=admin&pass=1234 | Visible in URL | Low |
| POST | /login | Hidden in body | Moderate |

**Questions:**

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

**Questions:**

1. Identify missing closing tags.
2. Analyze issues with the <img> tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

**Questions:**

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

**Code of Conduct:**

I Chunduri Naga Sai Srikanth (192311159)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

**Signature of the student:**

Chunduri Naga Sai Srikanth

**RUBRICS FOR CALCULATION:**

| Criteria | Weightage | Level 4 (Exemplary) | Level 3 (Proficient) | Level 2 (Developing) | Level 1 (Beginning) |
|---|---|---|---|---|---|

| | | | | | |
|---|---|---|---|---|---|
| Understanding of Data | 25% | Accurately interprets all data patterns, trends, and relationships | Interprets data correctly with minor gaps | Partial understanding; misses key trends | Misinterprets or fails to understand data |
| Analysis | 25% | Provides deep analysis with strong logical reasoning and justification | Provides reasonable analysis with some justification | Basic analysis with weak reasoning | No meaningful analysis provided |
| Application of Concepts | 20% | Effectively applies relevant concepts/tools to interpret data accurately | Applies concepts with minor errors | Limited application; requires guidance | Unable to apply concepts to data |
| Conclusion | 20% | Draws clear, meaningful conclusions with strong insights and implications | Provides valid conclusions with moderate insights | Conclusions are vague or weak | No clear or valid conclusions |
| Clarity | 10% | Presents interpretation clearly using proper structure, visuals, and terminology | Generally clear with minor issues | Lacks clarity or proper structure | Poor presentation and difficult to understand |

Answers

**1. HTTP Response Analysis**

1. Incorrect or invalid fields:

- Content-Length: -150 is invalid because content length cannot be negative.
- Other fields are generally valid.

2. Issue with Content-Length:

Content-Length specifies the response body size in bytes. A negative value is invalid and may cause parsing errors or communication problems.

3. Status code correctness:

200 OK indicates that the request was successful. However, the invalid Content-Length makes the response header incorrect.

4. Corrected header:

HTTP/1.1 200 OK

Date: Wed, 15 Oct 2026 07:14:00 GMT

Content-Type: text/html

Content-Length: 150

Connection: keep-alive

Note: 150 is an example; the actual value must match the response body size in bytes.

**2. DOM Structure Analysis**

1. Missing or improperly closed elements:

The  element is missing its closing  tag.

2. Browser interpretation:

Browsers usually automatically correct minor HTML errors and construct a DOM tree. However, the resulting structure may differ from the intended structure.

3. Impact on rendering and layout:

The browser may still display “Hello World,” but incorrect element nesting can affect page layout, styling, and JavaScript operations.

4. Corrected HTML code:

```
<html>
<head>
  <title>Test Page</title>
</head>
<body>
  <div>
    <p>Hello World</p>
  </div>
</body>
</html>
```

**3. GET vs POST in Login System**

1. Differences between GET and POST:

- GET: Sends data through URL parameters.
- POST: Sends data in the request body.
- GET data is visible in the URL, while POST data is not normally displayed in the URL.

2. Suitable method for login:

POST is more suitable for login because usernames and passwords are not included in the URL.

3. Security risks of GET:

- Passwords may appear in browser history.
- URLs may be stored in server logs.
- Sensitive data may be exposed through shared links or referrer information.

4. Best practices:

- Use POST for login requests.
- Use HTTPS to encrypt data during transmission.
- Hash passwords securely before storing them.
- Validate user input and use secure session management.
- Apply rate limiting to prevent repeated login attempts.

Conclusion: POST over HTTPS is the recommended approach for login systems.

**4. HTML Code Inspection**

1. Missing closing tags:

The <h1> and <p> elements are missing their closing tags.

2. Issues with the <img> tag:

The image tag is missing the alt attribute, which provides alternative text for accessibility and when the image cannot load.

3. Browser handling:

Browsers usually attempt to correct missing closing tags automatically. The page may display, but the HTML is not properly written.

4. Corrected HTML code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Sample</title>
</head>
<body>
  <h1>Welcome</h1>
```

```
    <p>This is a paragraph</p>
    <img src="image.jpg" alt="Sample image">
</body>
</html>
```

**5. Browser-Server Communication Flow**

1. Explanation of each step:

1. URL entry: The user enters a website address in the browser.
2. DNS lookup: DNS converts the domain name into an IP address.
3. HTTP request: The browser sends a request to the server.
4. Server processing: The server processes the request and prepares the response.
5. Response returned: The server sends an HTTP response containing a status code, headers, and possibly a body.
6. Page rendering: The browser processes HTML, CSS, and JavaScript to display the page.

2. Where delays can occur:

Delays may occur during DNS lookup, network connection, server processing, data transfer, and page rendering.

3. Role of DNS:

DNS helps the browser locate the correct server using the website's domain name.

4. Performance optimizations:

- Use DNS caching.
- Optimize server response time.
- Compress HTML, CSS, and JavaScript.
- Use browser caching and a Content Delivery Network (CDN).
- Optimize images and reduce unnecessary requests.

Conclusion: Optimizing network communication and page resources improves website loading speed.